Proof-to-Byte: Assumption-Minimal, Cross-Substrate Binding Assurance for PQ/T Hybrid KEMs

Anonymous Author(s)

***Abstract*—Post-quantum hybrid key-encapsulation (PQ/T KEMs—a lattice KEM combined with an elliptic-curve KEM) is being deployed across TLS, Signal, and MLS faster than its *deployment safety* can be assured: the same period has seen side-channel breaks (KyberSlash), binding / re-encapsulation breaks (PQXDH; Schmieg's ML-KEM malicious-key binding failures), and a widening gap between what a hybrid's *specification* guarantees and what its compiled, multi-platform *implementation* actually does. We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs built on one principle: *a security property is only as trustworthy as its weakest realization*.**

Its combiner, ContextBound, achieves *combiner binding* (MAL-BIND-K-CT and MAL-BIND-K-PK) reducing to collision-resistance of SHA3 *alone*—with no binding assumption on the component KEMs—machine-checked in EasyCrypt as a containerized, pinned-source hard CI gate. The *modeled* combiner is implemented once in a Rust `no_std` core. Its pinned output was checked on predecessor source in the deterministic OS/ISA conformance cells reported in this paper (four ISA targets executed—one under qemu-user—and one cross-compiled); the migrated source requires fresh execution. Native ABI 2 product adapters instead exercise signed-policy/context semantics, round trip, rollback, and failure atomicity without exposing caller-controlled seeds or raw combine operations. The artifact supplies a six-method conformance matrix and a self-validating source→binary constant-time probe configured to discriminate a zero ML-KEM secret path from a synthetic planted secret-indexed leak; its migrated-backend result is pending. A machine-checked *separation map* then pins down which combiner inputs are load-bearing, and where the lean X-Wing shape, instantiated over ML-KEM's expanded-`dk` serialization, is exposed while ContextBound is not—deployed seed-`dk` X-Wing is unaffected.

We are explicit about novelty: the binding *fact* follows from recent results; our contributions are this separation theorem and the *conjunction*—an assumption-minimal (for the combiner-binding property), machine-checked result tied by conformance (not mechanized refinement) to one implementation across a heterogeneous deployment surface, with reproducible gates that catch the failure classes that have broken deployed hybrids. A signed, monotonic policy is resolved into one authenticated suite/profile/key-format decision whose digest is absorbed into the application context. A machine-verifiable claim ledger keeps mechanized, conformance, device, and performance evidence separate. A source-bound, matched-backend single-host gate accepts only controlled runs whose proof digest matches the live canonical source tree. Authoritative JSON is strict-parsed from bounded regular-file snapshots, and selected-proof hashing and semantic validation consume the same bytes; release matrix and performance-budget policy are verifier-fixed, not proof-selected. Repository provenance ignores local Git excludes, rejects ignored Python bytecode, and dispatches verifier source under isolated/no-site CPython with a fresh private cache prefix; this is canonical source-input consistency after fixed generated-prefix exclusions, not a hermetic build-input, interpreter, or host attestation. Specification-to- implementation refinement, a current clean two-device Apple matrix, and cross-device/end-to-end performance parity remain explicitly pending. The current research-alpha release graph uses target-selected `mlkem-native` v1.2.0 through a dedicated Rust/C boundary, plus pinned `fips204` 0.4.6 and `sha3` 0.10.9. Exactly five little-endian AArch64 Apple/Linux/Android targets use upstream native arithmetic plus a fixed Armv8-A FIPS 202 assembly profile; every other target, including Wasm, remains portable C. There is no runtime dispatch or Armv8.4-A SHA3 path, and ABI 2, key, and wire contracts are unchanged. That migration changed the canonical source digest, so all portable-derived package, device, performance, and binary-CT results are historical evidence; no clean source-bound replacement timing, device, or binary-CT measurement is claimed here. The current-source local footprint capture reported below is a platform/toolchain-specific diagnostic, not release provenance. ABI `2 / 0.1.0` is the stable-version, pre-publication package-ready Rust-crate line. Its package contract alone does not prove crates.io upload-API acceptance, crate-name ownership, publishing credentials or authorization, server-side policy acceptance, or a registry receipt. Receipt-gated, attestation-bound stable GitHub targets cover an Apple XCFramework revision and an Android/Linux platform distribution; the unsigned Windows output remains an unsupported CI diagnostic outside that asset set. No unverified registry publication, code-signing beyond the Apple lane, physical-device coverage, or production release is claimed.

***Index Terms*—Post-quantum cryptography, hybrid key encapsulation, binding/committing security, formal verification, constant-time, cross-platform assurance, TLS.**

Submitted to IEEE TDSC; author and affiliation omitted for double-blind review. An anonymized, history-free review artifact must be supplied through the editor; its public URL is withheld to preserve anonymity.

I. Introduction

THE migration to post-quantum (PQ) key establishment is being done conservatively, through *hybrids*: a PQ KEM (ML-KEM [1]) run alongside a classical KEM (X25519), with the two shared secrets combined so that the session key is secure if *either* component holds. Hybrid KEMs are now shipping in TLS 1.3 (`X25519MLKEM768`), in Signal's PQXDH and subsequent Triple Ratchet, and in MLS; designs such as X-Wing are also proposed in individual Internet-Drafts [5].

The gap this paper addresses: Hybrids are being deployed faster than their *deployment safety* can be assured, and the failures of the last two years were realization-level, not failures of the underlying hard problems. *(i) Side channels.* KyberSlash [6] showed that secret-dependent timings in widely-used Kyber/ML-KEM implementations leaked the secret key—a property of compiled code that a source-level constant-time argument does not by itself settle. *(ii) Binding.* The PQXDH protocol admitted a re-encapsulation issue, and Schmieg [3] proved that ML-KEM, in the FIPS-203 *expanded* decapsulation-key form many libraries store, is neither MAL-BIND-K-CT nor

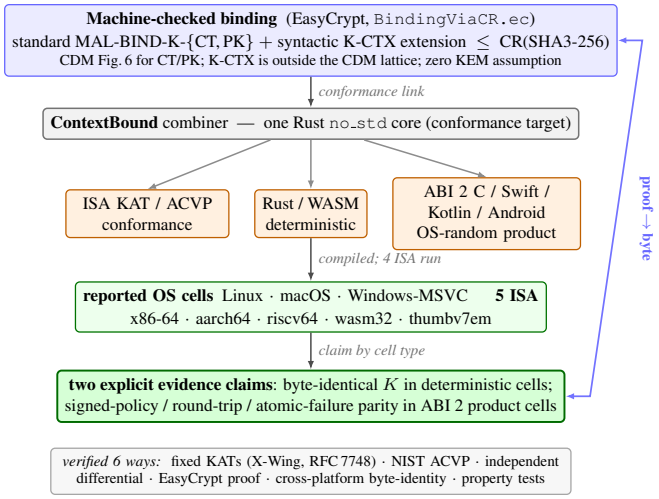

Fig. 1. Proof-to-byte. One machine-checked binding result (top) is tied by conformance—not a verified refinement proof—to a single Rust `no_std` core. Deterministic cells converge on a pinned shared secret K; native ABI 2 product cells instead verify fail-closed semantic parity without a deterministic product bypass.

MAL-BIND-K-PK—so whether a hybrid binds its transcript can depend on a serialization choice made three layers down. *(iii) Spec-to-binary drift.* A hybrid's guarantee is stated on a specification, sometimes proved on a model or source, but *run* as a compiled artifact—and real deployments compile it through many language bindings, operating systems, and instruction sets, each an opportunity for the specified model and the executed object to diverge. These three are one underlying problem: *a security property holds only of a realization, and a hybrid has many*. This is distinct from—and orthogonal to—*auditability* and *crypto-agility* [18], which describe *what* crypto is running; the question here is whether the hybrid that actually runs is *safe*.

Approach: We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs built around one principle: a security property is only as trustworthy as its weakest realization. Figure 1 is the organising idea—a single machine-checked binding result tied by conformance to one core implementation across the reported deployment surface ("proof-to-byte"), guarded by reproducible gates (configured in CI; we report constant-time results here as local runs) that target the specific failure classes above.[1]

Contributions:

- **C1 (Section III).** An *assumption-minimal, machine-checked commitment / transcript-binding* result: ContextBound's session key commits to the full transcript, reducing the standard MAL-BIND-K-CT/-K-PK notions and a self-defined, context-parameterized -K-CTX syntactic extension to collision-resistance of SHA3 *alone*, with the injective encoding proved. The standard axes instantiate the Cremers–Dax–Medinger Figure-6 game; K-CTX reuses its rejection skeleton but is outside the published CDM lattice. We are explicit (Section III) that the cryptographic content is this injective-encoding + CR(SHA3) argument—the component Decaps is *inert*, which is exactly why no KEM assumption is needed—and that C1's value is its completeness, mechanization, and role as the formal object tied by conformance to C2, not proof difficulty. On the same kernel we additionally prove a machine-checked *separation map* (Theorem 1, Table IV): deleting each combiner input in turn, we show that omitting pk_{pq} loses MAL-BIND-K-PK over the expanded-`dk` serialization (concrete $\Pr = 1$ adversary) but not over seed-`dk`; omitting context from our *context-parameterized wrapper* fails the locally named K-CTX collision game with $\Pr = 1$; and omitting ct_{pq} is *conditionally* safe in the modeled J-injective setting (the shared secret already binds it). Thus public-key absorption is necessary over expanded-`dk`, context absorption is necessary only for that explicit wrapper property, and ciphertext absorption is redundant only under the stated injectivity. The X-Wing field shape, instantiated over the FIPS-203 expanded-`dk` form, loses standard K-PK while keeping K-CT; a correctly deployed seed-`dk` X-Wing (which its individual draft mandates) reaches MAL-BIND-K-{CT,PK} (Section V-B). Native X-Wing has no context parameter, so K-CTX is not a well-typed comparison; only a hypothetical context wrapper around that shape which ignores its context fails our local wrapper game.
- **C2 (Section IV).** *Proof-to-byte cross-substrate conformance*: one core implementation has deterministic Rust/WASM/ISA harnesses; predecessor-source evidence *executed* 4 ISAs (3 native + riscv64 under qemu-user) and cross-compiled with by-construction identity on 1 more. The backend-migrated source must rerun those cells before they are reported as current. Native ABI 2 C/Swift/Kotlin/Android cells reuse the same core but test semantic product invariants rather than caller-selected randomness. The artifact keeps the hardware-dependent Apple check separate: a source-bound Xcode lane can run the Swift/C-ABI face on paired physical iOS/iPadOS devices; a separate Android lane runs the AAR/JNI face under ART. Device results are source-bound diagnostic evidence, not package-distribution claims, and are never inferred from a host package build.
- **C3 (Section V).** Reproducible assurance *instruments* wired into CI (the EasyCrypt binding proof is re-checked as a *containerized pinned-source hard gate* in a versioned container lane, as are the constant-time/regression oracles; Tamarin/ProVerif lemma/query presence and full prover execution are hard-gated in CI, stated precisely in Section V): a self-validating source→binary constant-time probe that requires a zero genuine-secret ML-KEM result against a positive synthetic planted-leak control. The target-selected `mlkem-native` probes need fresh x86-64-portable/aarch64-native captures; the intervening `fips203` provider failed on both ISAs, while former

[1]To be precise about the link: the EasyCrypt proof is over an abstract injective `encode`, and the shipped Rust `encode` is tied to it by *conformance* (shared ContextBound KATs + the six-method matrix of Section IV), not by a mechanized refinement proof. "Proof-to-byte" thus names a conformance-established link, not a verified compiler. The claim ledger records this as conformance evidence and intentionally leaves specification-to-implementation refinement open.

`libcrux` zero and older-source PQClean-HQC results are historical, not a current gate. An executable *regression oracle* covers the known dk-format binding coupling; and two independent symbolic provers. We report the constant-time measurements as local runs.

- **C4 (Section VI).** A private-use research TLS 1.3 integration against the production rustls library (`CryptoProvider`), with historical predecessor-source `tc netem` evaluation and no current source-bound performance claim.
- **C5 (Section IV).** A fail-closed policy-to-byte path: an ML-DSA-authenticated, strictly parsed, monotonic policy is atomically resolved to a closed suite/profile/key-format decision; the policy version and digest are carried in trusted state, and the digest is absorbed into a domain-separated application context. The signed-policy native ABI 2 and WASM execution APIs accept this atomic decision rather than independently re-selecting its fields. Native ABI 2 exports no raw hybrid, deterministic key-generation, X-Wing, or combine operation; its 40-byte decision descriptor and WASM's raw/conformance inputs remain forgeable trusted-caller values—not authorization capabilities—and are excluded from the authentication claim. A hostile local caller requires a service/process that owns the pinned verification key and monotonic state; an opaque handle in the same address space is insufficient.

What is and is not novel: We state the boundary plainly. *Not novel, and attributed:* that a "hash-everything" combiner is binding from collision-resistance alone while a lean combiner inherits the omitted component's binding is the Chempat result [4]; ContextBound *is* that construction, not a new primitive. That ML-KEM's expanded-dk form breaks binding (and the seed form fixes it) is Schmieg's [3]. The binding-notion lattice is CDM's [2]. That a constant-time harness must mark only secret sub-fields is standard practice [6]. *Artifact-specific contribution:* first, a *machine-checked shape* × *format separation* (Theorem 1)—where Schmieg shows a *single* KEM's binding depends on its `dk` format, we prove, in one model, the orthogonal statement that the *combiner shape* determines whether that format-dependence survives composition (lean: yes—broken over expanded-`dk`, binding over seed-`dk`; full: no—binding over both), a result instantiated and checked in our model. We do not assert exhaustive priority over adjacent generic-combiner or safe-omission work without a complete primary-source review. Second, the *conjunction* C1∧C2—a CR-only, machine-checked binding result tied by explicit conformance tests to one implementation across a heterogeneous substrate; third, the self-validating source→binary *discriminator* as a reusable CI artifact. The design invariant that binding the ciphertext and public key in the KDF makes hybrid binding robust to the component KEM's key-serialization format is thus no longer merely operationalized but *proven*. We do **not** claim stronger binding than X-Wing (Section III), nor any live exploited deployment. Table I makes the conjunction concrete: this artifact ties a mechanized hybrid-combiner binding theorem to a multi-substrate conformance matrix with source→binary and TLS gates. The table reports capabilities demonstrated by the cited comparison artifacts, not an exhaustive priority search—and we are equally explicit that an axis we do *not* hold (a verified specification-to-binary implementation) is held by formosa-mlkem.

TABLE I
ARTIFACT-POSITIONING DIMENSIONS. ✓ DENOTES AN ARTIFACT CAPABILITY DEMONSTRATED IN THIS PAPER, NOT DEPLOYMENT MATURITY, STANDARDIZATION STATUS, OR AN EXHAUSTIVE PRIORITY SEARCH; ∼ PARTIAL; BLANK: NOT DEMONSTRATED IN THE SCOPED CITED ARTIFACT. THE CONTRIBUTION IS THE CONJUNCTION (LAST COLUMN), NOT ANY SINGLE ROW.

	X-Wing	formosa	SandboxAQ	Chempat	**this work**
Mechanized binding (combiner)			∼		✓
Both rejection styles, $K \neq \perp$				∼	✓
Zero KEM-binding assumption				✓	✓
Byte-identical multi-substrate object					✓
Source→binary CT gate		∼			✓
rustls TLS 1.3 integration	∼				✓
Verified spec→binary impl. *(we do not)*		✓	∼		

II. Background and Threat Model

A. PQ/T hybrid KEMs and combiners

ML-KEM [1] is the standardized form of CRYSTALS-Kyber [10]; like most lattice KEMs it applies a Fujisaki–Okamoto transform [12], [13] with *implicit rejection* (a malformed ciphertext yields a pseudorandom key rather than $\perp$). The companion signature standards are ML-DSA [8] and SLH-DSA [9], and the broader process is surveyed in [11]. Hybrid key exchange in TLS 1.3 [15] follows the IETF framework [16]—e.g. the `X25519MLKEM768` group [19] over X25519 [14]—and deployed PQ/T protocols include Signal's PQXDH [20] plus SPQR/Triple Ratchet [22], [23], and Apple's PQ3 [21].

A hybrid KEM combines component shared secrets ss_{pq} and ss_{tr} into a session key K. The design space ranges from simple *concatenation* KDFs (`X25519MLKEM768`) to combiners that additionally absorb ciphertexts and public keys. X-Wing [5] uses a fixed "lean" combiner $K = \mathrm{SHA3}(ss_{\mathrm{pq}} \,\|\, ss_{\mathrm{tr}} \,\|\, ct_{\mathrm{tr}} \,\|\, pk_{\mathrm{tr}} \,\|\, \mathrm{label})$, omitting the ML-KEM ciphertext and public key.

B. KEM binding

We use the Cremers–Dax–Medinger (CDM) X-BIND-P-Q framework [2]. A *transcript* is a tuple (pk, ct, K) produced by a (possibly malicious) execution; the two "observable" quantities a binding notion may constrain are the public key PK, the ciphertext CT, and the derived key K. For $P, Q \in \{K, \mathsf{PK}, \mathsf{CT}\}$, a KEM is X-BIND-$P$-$Q$ if no adversary in the game below wins:

Game $\mathsf{Bind}^{P,Q}_{\mathcal{A}}$: the adversary $\mathcal{A}$ outputs two transcripts (pk_0, ct_0, K_0) and (pk_1, ct_1, K_1). $\mathcal{A}$ *wins* iff $P_0 = P_1 \;\wedge\; Q_0 \neq Q_1 \;\wedge\; K_0 \neq \perp \;\wedge\; K_1 \neq \perp$.

Three adversary classes parameterize how the key material in each transcript is produced, in increasing strength: **HON** (honest KeyGen), **LEAK** (honest keys, secret keys leaked to $\mathcal{A}$), and **MAL** (adversarially generated keys). Thus MAL $\supseteq$ LEAK $\supseteq$ HON, and CDM prove monotonicity across the (P, Q) lattice, so MAL-BIND-K-CT and MAL-BIND-K-PK jointly yield the whole K-binds-{PK,CT} sub-lattice. For implicitly-rejecting KEMs such as ML-KEM, these two are the achievable ceiling on those axes; the dual X-BIND-CT-$*$ notions (a ciphertext binding the key or public key) are *structurally unachievable*, since decapsulation never returns $\perp$ and a mutated ciphertext always yields *some* key. The binding of implicitly-rejecting KEMs—and how it fails under malicious, expanded-form keys—is studied in detail by Schmieg and others [3], [26].

Our K-CTX label is deliberately separate. In the standard CDM syntax, CTX is a caller input, not a transcript output in $\{K, PK, CT\}$, so it is neither a CDM lattice node nor a consequence of CDM monotonicity. We define a context-parameterized KEM whose Encaps/Decaps interfaces both take the context, then apply the same accepting/rejecting game skeleton to the key/context projection. This hash-level statement says that a colliding accepted key cannot accompany different context bytes; it does not authenticate what those bytes mean. Authenticated context agreement is a separate protocol property of the Tamarin/ProVerif models and the signed-policy execution boundary.

C. Threat model

Reflecting the deployment-safety framing, we consider three realization-level adversaries, each matched to one assurance instrument.

The malicious-key (MAL) binding adversary (Section III) is the strongest CDM class on the standard CT/PK axes, with an analogous adversary in our explicitly labeled K-CTX extension: it generates *all* key material, including mutually inconsistent or maliciously-serialized keys, and wins if it produces two accepting executions whose hybrid keys collide while a ciphertext, public key, or context differs. This is the adversary behind re-encapsulation attacks (PQXDH) and the ML-KEM expanded-key binding failures; it is the one our combiner's binding theorem targets, and the one the dk-format demonstration of Section V exercises concretely.

The binary-level constant-time adversary (Section V) observes wall-clock or microarchitectural timing and seeks any secret-dependent conditional branch, memory index, or division in the *compiled* code—not the source. KyberSlash is the canonical instance. Source -level constant-time arguments do not discharge this adversary on their own, which is why we probe the emitted binary.

The downgrade/agility adversary attacks profile and policy negotiation, attempting to force a weaker suite or combiner. We bind a signed, versioned policy and a domain-separating `policy_version` into the transcript (Algorithm 1) and fail closed on an unsatisfiable policy; a full treatment of agility is out of scope here.

We *assume*, rather than re-prove, the confidentiality of the component primitives—IND-CCA2 of ML-KEM and the Diffie–Hellman assumption for X25519—and the collision-resistance of SHA3; our contribution is orthogonal, concerning whether the *realization* preserves the binding and constant-time properties the primitives are chosen for.

III. ContextBound and the Machine-Checked Kernel

A. Construction

ContextBound derives

$$K = \mathrm{SHA3\text{-}256}\big(\mathrm{encode}[\mathsf{LABEL}, \mathsf{suite\_id}, \mathsf{policy\_version}, ss_{\mathrm{pq}}, ss_{\mathrm{tr}}, ct_{\mathrm{pq}}, pk_{\mathrm{pq}}, ct_{\mathrm{tr}}, pk_{\mathrm{tr}}, \mathsf{context}]\big),$$

where `encode` concatenates each field under a fixed-width 8-byte big-endian length prefix (Algorithm 1). Two design choices are load-bearing. *(i) Injective, length-prefixed encoding.* A naive concatenation $a \,\|\, b$ is ambiguous—("ab", "") and ("a", "b") collide—which is precisely the structure a binding adversary exploits. Prefixing every field with its fixed-width length makes the encoding self-delimiting and therefore injective (proved, Section III), so distinct transcripts map to distinct byte strings and the binding question reduces cleanly to collision-resistance of the hash. *(ii) Absorb everything.* ContextBound feeds *all* component ciphertexts and public keys (and an application context, and a domain-separating label, suite identifier, and policy version) into the hash. By the combiner-binding theorem of Chempat [4], the binding advantage is then bounded by $\mathrm{Adv}^{\mathrm{CR}}$ alone, with no residual term requiring any component KEM to be binding—trading a KEM-self-binding assumption for a single, well-studied collision-resistance assumption. The dual profile `CompatXWing` reproduces X-Wing's lean combiner byte-for-byte for construction compatibility and therefore binds *only* $ss_{\mathrm{pq}} \,\|\, ss_{\mathrm{trad}} \,\|\, ct_{\mathrm{trad}} \,\|\, pk_{\mathrm{trad}}$: any suite identifier, policy version, or context is absent from the X-Wing construction. Q-Periapt requires their canonical absence (empty, zero, empty) and rejects supplied values instead of silently discarding them; these fields are bound *exclusively* by `ContextBound`, never by `CompatXWing`. `ContextBound` is the hash-everything profile and is policy-selected when an application needs context binding or wishes to avoid the serialization-format dependence of Section V. The suite is open source.[2]

B. The EasyCrypt development

Figure 2 shows the mechanized reduction. The encoding's injectivity, `encode_inj`, is a *proved lemma*, not an assumed one. The proof has two steps. A field-level lemma, `lp_cat_inj`, shows that a single length-prefixed field is self-delimiting: if $\mathsf{lp}(a) \,\|\, x = \mathsf{lp}(b) \,\|\, y$ then $a = b$ and $x = y$, because the two 8-byte prefixes have equal width and (being injective) force $|a| = |b|$, after which list-concatenation cancellation (`eqseq_cat`) splits off equal field bodies and equal remainders. A structural induction over the field list then lifts this to whole encodings: a non-empty encoding has size ≥ 8 and so can never equal the empty one, ruling out

[2]Anonymized artifact repository provided to reviewers via the editor.

Algorithm 1: CONTEXTBOUND combiner. `CompatXWing` omits $ct_{\mathrm{pq}}, pk_{\mathrm{pq}}, S, v, x$ and uses X-Wing's label (byte-exact X-Wing; binding over the deployed seed-`dk` serialization, cf. Theorem 1).

Input: label L, suite id S, version v, secrets $ss_{\mathrm{pq}}, ss_{\mathrm{tr}}$, ciphertexts $ct_{\mathrm{pq}}, ct_{\mathrm{tr}}$, public keys $pk_{\mathrm{pq}}, pk_{\mathrm{tr}}$, context x
Output: 32-byte shared secret K
$\mathsf{lp}(f) \leftarrow \mathsf{be8}(|f|) \,\|\, f$ `// 8-byte big-endian length prefix`
$T \leftarrow [\,L,\ S,\ \mathsf{be4}(v),\ ss_{\mathrm{pq}},\ ss_{\mathrm{tr}},\ ct_{\mathrm{pq}},\ pk_{\mathrm{pq}},\ ct_{\mathrm{tr}},\ pk_{\mathrm{tr}},\ x\,]$
$e \leftarrow \mathsf{lp}(T_0) \,\|\, \mathsf{lp}(T_1) \,\|\, \cdots \,\|\, \mathsf{lp}(T_{n-1})$ `// injective encode`
return SHA3-256(e)

boundary-shift collisions, and the inductive step peels one field with `lp_cat_inj`. The whole argument bottoms out at two elementary facts about the length field—that an 8-byte big-endian integer occupies 8 bytes (`be8_size`) and is injective (`be8_inj`). On top of `encode_inj` we discharge (i) a generic transcript-collision reduction `bind_le_cr`: a `byequiv` argument that maps any adversary winning the binding game to one finding an H-collision, since a differing observable forces a differing field list (hence, by injectivity, a differing encoding) while the colliding key forces equal hashes; and (ii) the *KEM-aware* game, in which the MAL adversary supplies the keypairs and K is *derived* via the component Decaps functions and the combiner. We mechanize both the implicit-rejection specialization (`malbind_*_le_cr`) and explicit rejection (`malbind_*_xrej_le_cr`), where the $K \neq \perp$ side condition is *present and load-bearing*—an explicit probability-one countermodel demonstrates the game failure if it is removed. These are CDM Figure-6 instantiations for CT/PK and the same rejection skeleton over the context-parameterized syntax for K-CTX. Every theorem reduces only to CR(SHA3); the development compiles under the EasyCrypt checker (we pin the checker and SMT-solver versions in the artifact), and proof-dependency regression controls ensure the current scripts use `encode_inj` and their axis-specific disagreement lemmas.

What this is, precisely. We state the cryptographic content plainly, because a CDM-literate reader should not over-read the label. Since K *is defined* as $H(\mathrm{encode}[\ldots])$ and the tuple contains the ciphertexts and public keys *directly*, the proof is in substance a *commitment / transcript-binding* statement: it shows that one cannot collide K while changing a ciphertext, public key, or context, by reducing to CR(H) over the proved injective encoding. The component Decaps—and hence the adversary's freedom over key material that CDM binding is fundamentally about—is *inert* in the argument: the theorems would hold for *any* Decaps, which is exactly what "zero KEM assumption" means and why it is achievable. The value of C1 is therefore its *completeness* (both rejection styles, with $K \neq \perp$ discharged), its *mechanization*, and its role as the formal object tied by conformance—not mechanized refinement—to the substrate of Section IV; it is not the intrinsic difficulty of the collision reduction. The concrete attack-resistance arguments (re-encapsulation, the dk-format coupling of Section V) are *design arguments* consistent with this commitment property, not claims the EasyCrypt artifact witnesses at the Decaps level.

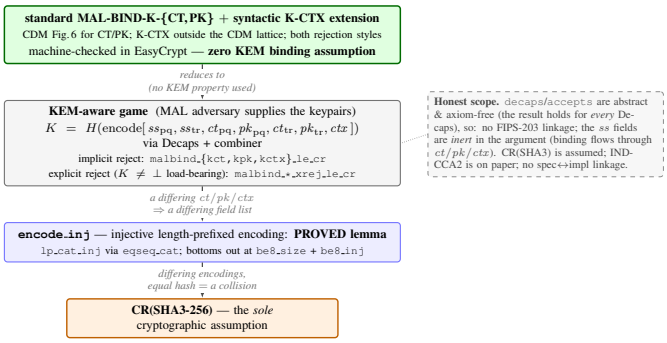

Fig. 2. The kernel as a reduction tower: standard MAL-BIND-K-{CT,PK} plus the separately labeled syntactic K-CTX extension (both rejection styles) reduce—using no property of Decaps—through the proved `encode_inj` to collision-resistance of SHA3, the sole cryptographic assumption.

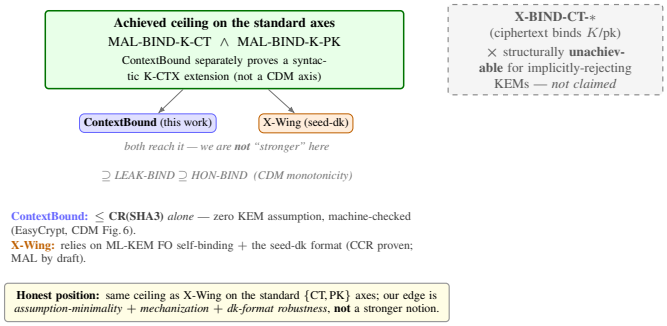

Fig. 3. Binding position. Both schemes reach the standard ceiling at a fixed serialization; our edge is assumption-minimality, mechanization, and a machine-checked dk-format robustness separation (Theorem 1)—not a stronger notion on a shared axis. X-BIND-CT-∗ is structurally unachievable for implicitly-rejecting KEMs and is not claimed.

C. Binding position

Figure 3 states our position precisely. On the standard {CT,PK} axes, ContextBound and a correctly-implemented seed-dk X-Wing both reach the MAL-BIND-K-{CT,PK} ceiling; we are *not* "stronger" there. Our edge is (a) *assumption-minimality*: ContextBound's binding reduces to CR(SHA3) alone, whereas X-Wing's relies on ML-KEM's Fujisaki–Okamoto self-binding together with the seed-dk format; (b) *mechanization*; and (c) *provable dk-format robustness* (Theorem 1)—and here the distinction is sharp: while we are not stronger on the {CT,PK} axis *at a fixed* serialization, the lean shape's MAL-BIND-K-PK *depends on* the format (it is broken over expanded-`dk`), whereas ContextBound's holds over *both*. That is a genuine, machine-checked robustness separation, not a stronger notion on a shared axis. Scope of the mechanization: Decaps is modelled as an abstract function (which is exactly why "zero KEM assumption" is literal), so there is no FIPS-203 linkage and the shared-secret fields are inert in the binding argument; CR(SHA3) is assumed; IND-CCA2 robustness is argued on paper; and there is no specification-to-implementation linkage proof.

IV. Proof-to-Byte Cross-Substrate Realization

A. Architecture

The combiner and its encoding live in a single dependency-free Rust `no_std` core; the component primitives are *injected* through narrow traits (a `Kem` trait for ML-KEM and X25519, an extendable-output `Xof` trait for the hash), so the implementation under conformance test—the encoding and combiner—is the same code regardless of which third-party backend (target-selected `mlkem-native` v1.2.0 for ML-KEM, x25519-dalek, and `sha3` 0.10.9) is plugged in; ML-DSA uses `fips204` 0.4.6. Backend correctness, audit, and constant-time posture remain per implementation and ISA. The ML-KEM source is commit- and archive-hash-pinned, but RustSec does not inspect vendored C and neither the provider nor this Rust/C integration has completed an independent audit. Exactly the little-endian targets `aarch64-apple-darwin`, `aarch64-apple-ios`, `aarch64-apple-ios-sim`, `aarch64-unknown-linux-gnu`, and `aarch64-linux-android` use upstream native arithmetic and fixed Armv8-A scalar x1/scalar-Neon x4 FIPS 202 assembly, while all other targets use portable C. The selection is compile-time only, with no runtime dispatch or v8.4-A SHA3 path. Upstream HOL-Light evidence covers only selected upstream assembly source/object routines, not downstream reassembly, this wrapper, or the full ABI. This trait boundary is also where *crypto-agility* lives: `Kem::C2PRI` records the primitive property, while the separate, default-false `Kem::COMPAT_XWING_SAFE` capability records whether the exposed key format preserves X-Wing's seed-dk self-binding precondition. The lean profile requires both constants, so an inconsistent third-party declaration fails closed. A signed, versioned *policy* is strictly parsed and verified before the policy crate atomically resolves a closed suite/profile/key-format decision. Rollback is rejected, and reuse of a version with a different digest is treated as equivocation. The native ABI 2 and signed-policy WASM paths consume that decision and absorb its policy digest into a domain-separated application context. Signed-policy documents and policy-bound application contexts are each capped at 64 KiB before Rust-side verification, parsing, or derived-context allocation; Java/Kotlin/JNI also check before their explicit native copies, while host-runtime marshalling and service-level quotas remain outside this bound. Oversized inputs fail explicitly. The rustls demonstrator is intentionally separate: from an already parsed but not cryptographically authenticated `Policy`, it accepts only the ML-KEM-768 + X25519 suite at policy version 1, selecting either ContextBound/expanded-key or CompatXWing/seed-dk exactly as resolved. It does not consume an authenticated decision, policy digest, or monotonic store. Its fixed protocol-domain context demonstrates fail-closed suite/profile selection, but not signed-policy authorization or arbitrary per-session K-CTX binding. The core's owned shared-secret storage is zeroized on drop, and the concrete SHA3 staging owner marks public and secret absorptions without changing their byte stream: it volatile-wipes component-secret and conservatively sensitive caller-context ranges from both inline and heap copies and fails closed to a whole-buffer wipe for unclassified input or invalid range metadata. This is local storage hygiene, not a full-runtime erasure claim. The composition code is written in a branchless, mask-based constant-time style (Section V); arrays copied across Swift/JVM/JavaScript/OS boundaries remain caller-managed best effort rather than a full-stack zeroization guarantee.

The core is exposed through deterministic Rust/WASM conformance surfaces and native ABI 2 C, Swift, Kotlin, and Android/JNI product adapters. Each is a thin shim that marshals bytes in and out of the same core. Deterministic cells can answer whether a realization produces pinned bytes; OS-random product cells instead answer whether it preserves the authenticated decision, round-trip, context-separation, state-transition, and failure-atomicity semantics. These are complementary, not interchangeable, evidence claims.

B. The six-method conformance matrix

Table II lists the six orthogonal methods (NIST ACVP vectors [37] among them) that answer this. They are deliberately heterogeneous in their oracle and in what they catch, so a defect that escapes one is caught by another. Table III reports predecessor-source substrate coverage with per-cell status: the byte-identical shared secret was run natively on x86-64 and aarch64, on a WebAssembly runtime (wasm32), and under `qemu-user` emulation on riscv64. The bare-metal no_std `thumbv7em` target was only cross-compiled. The backend migration invalidated those source-bound cells for release promotion; current-source execution must be reported separately.

Current-source local footprint (platform/toolchain-dependent; regenerate per host with `paper/footprint.sh`; exact rows in `paper/footprint.csv`). On Darwin 27.0.0 arm64 with Rust 1.96.1, `wasm-pack` 0.15.0, and Homebrew LLVM Clang 22.1.8, the stripped native ABI 2 shared library—including ML-KEM-768, X25519, the combiner, signed-policy ML-DSA-65 verification, and OS-random product entry points—is 667.8 KiB (683,800 bytes); the lean WebAssembly module is 97.7 KiB (100,034 bytes), and the optional `signed-policy` module is 332.3 KiB (340,298 bytes). These are local current-working-source diagnostics, not clean signed provenance, architecture-independent package sizes, or a cross-platform binary claim. Separately, the dependency-free core cross-compiles to the bare-metal `thumbv7em` target. This establishes predecessor-source buildability on that target, not current package/runtime evidence from an embedded MCU.

V. CI-Gated Assurance

A. A *source→binary constant-time* discriminator

The release-graph ML-KEM backend is now target-selected `mlkem-native` v1.2.0 behind a dedicated safe Rust/C boundary. No source-level secret-independence result from a replaced provider transfers to this backend. The binary-level question—does the compiled path branch or index on the genuine secret?—is tested by a probe that marks only the genuinely-secret decapsulation-key sub-fields ($\hat{s}$, z) for each parameter set and runs real `mlkem-native` decapsulation under Memcheck. The harness is self-validating: a synthetic planted secret-indexed

TABLE II
THE SIX ORTHOGONAL VERIFICATION METHODS.

Method	Reference / oracle	Independence	What it catches
Fixed KATs	X-Wing draft vectors; RFC 7748	published vectors	combiner and X25519 spec-conformance regressions
NIST ACVP	*usnistgov* ACVP, wired FIPS parameter sets/modes	NIST ground truth	primitive non-conformance (FIPS 203/204/205); ML-DSA internal-interface vectors remain unwired reference data
Multi-backend differential	RustCrypto `ml-kem`/`ml-dsa`; orion X25519	independent re-implementations	implementation bugs (output must stay byte-identical)
EasyCrypt proof	machine-checked; CR(SHA3)	formal (computational)	binding-design flaws in the combiner
Cross-platform realization	deterministic vectors + product invariants	evidence mode separated per cell	byte divergence in conformance cells; policy/context/state/failure drift in ABI 2 product cells
Generative property tests	proptest: 9 properties × 128 cases	randomized	edge-case violations (injectivity, domain separation, K-CTX)

TABLE III
HISTORICAL PREDECESSOR-SOURCE CROSS-SUBSTRATE COVERAGE (✓ MEANS VERIFIED ON THAT SOURCE, NOT ON THE MIGRATED RELEASE SOURCE). EVERY SOURCE-BOUND CELL MUST BE RERUN.

(a) ISA targets (Rust core; the byte-level claim)

ISA	runner	build	byte-id. K	binary-CT
x86-64	Linux, Win-MSVC	✓	✓ (run)	historical
aarch64	Linux, macOS	✓	✓ (run)	historical
wasm32	node	✓	✓ (run)	not checked
riscv64	qemu-user	✓	✓ (run)‡	not checked
thumbv7em	bare-metal	✓	by constr.†	n/a

‡ executed under `qemu-user` emulation (reference-vector KATs reproduce pinned bytes); † cross-compiled, identity by FIPS-deterministic encodings + identical source.

(b) native ABI 2 product workflow (release receipts / device evidence)

Face	Linux	macOS	Windows	device
C ABI	released§	historical	released§	n/a
Swift	n/a	released§	n/a	iOS historical
Kotlin (JVM/FFM)	n/a	historical	n/a	n/a
Android/JNI	n/a	n/a	n/a	emulator§

§ "released" denotes the immutable, attestation-bound GitHub *research prereleases*: the Apple XCFramework revision and the Android AAR (with API 35 / 16 KiB-page emulator runtime evidence) plus Linux x86_64/aarch64 and unsigned experimental Windows x64 MSVC SDK archives, each validated by attested candidate-CI native consumers and bound to machine-checked release receipts. A clean-tree schema-v3 physical iPad+iPhone matrix passed for predecessor source only; the target-selection migration made it stale, and both physical lanes still require same-source native reruns. All "released" cells are exact pre-selection historical artifacts; their receipts do not attest a target-selected rebuild. No registry publication or physical-Android coverage is claimed.

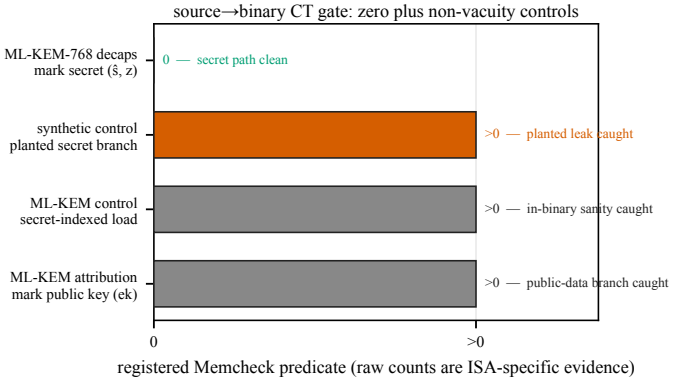

Fig. 4. Historical predecessor-source contrast: the `libcrux` ML-KEM secret yielded zero flags while the retired PQClean-HQC path's constant-weight sampler yielded 193. The target-selected `mlkem-native` gate retains the synthetic planted control but requires fresh x86-64-portable/aarch64-native captures; the intervening `fips203` run failed and is historical.

load must report positive. Whole-dk and embedded-public-key observations are diagnostic only because an expanded FIPS-203 dk embeds public ek and $H(ek)$ fields.

No fresh target-selected binary-CT result is claimed in this paper. The former `libcrux` aarch64 zero and embedded-public-key 5696-report captures, together with the now-retired PQClean-HQC backend's **193** aarch64 and 22,849 x86-64 counts, are historical predecessor-source evidence. The intervening `fips203` 0.4.3 provider also failed the corrected gate: 34,306 errors across 100 contexts on x86-64 and 30,464 across 70 contexts on aarch64 in CI run 29230650107[3]. Those counts are historical `fips203` failure evidence, not a result for `mlkem-native`. Pre-selection portable `mlkem-native` captures are historical too. Production promotion of the ABI 2 binary surface requires fresh x86-64-portable and aarch64-native runs whose genuine-secret probes report zero and planted controls report positive.

Why marking granularity matters. A naive harness that marks the *whole* serialized decapsulation key secret produces, on the aarch64 NEON path, 5696 reports across 60 branches comparing 12-bit coefficients to q—which look exactly like a KyberSlash-class leak. They are not: the FIPS-203 `dk` *embeds* the public key ($\texttt{dk} = \texttt{dk\_pke} \,\|\, \texttt{ek} \,\|\, H(\texttt{ek}) \,\|\, z$), and those branches are the compiler's scalar lowering of the public-key reduction run during FO re-encryption—operating on *public* bytes. Marking only the genuinely-secret sub-fields ($\hat{s}$ and the implicit-rejection seed z) yields zero reports. This is standard practice [6], but the embedded-public-key pitfall is a real trap; our probe encodes the correct granularity and gates it.

[3]https://github.com/billlza/q-periapt/actions/runs/29230650107

B. A mechanized field-separation map

Section III shows ContextBound *attains* the binding ceiling. Here we prove the paper's distinctive cryptographic contribution: a *machine-checked map* pinning down, field by field, *which inputs the combiner must absorb*—and *why*—across the standard CDM CT/PK axes and the separately identified K-CTX syntactic extension. K-CTX is not a third CDM lattice axis and no CDM monotonicity result is claimed for it. All of it lives in *one* EasyCrypt model (a single execution type, a single shared-secret wiring $ss = J(z, ct)$), so every verdict is attributable to combiner *structure*, not to a difference in the modeled KEM. Starting from the full ContextBound field list (which absorbs ct_{pq}, pk_{pq}, and the context) we delete *exactly one* field and ask which binding notion the deletion costs—*three distinct combiners*, not one combiner read under three notions.

Theorem 1 (separation map, machine-checked)**.** *Model $ss = J(z, ct)$ (FIPS-203 §6.3). In one binding model:*

1) Omit pk_{pq} $\Rightarrow$ lose K-PK, serialization-conditionally. *Over expanded-*`dk` *(z adversary-substitutable) a concrete deterministic adversary wins MAL-BIND-K-PK with probability* 1 *(*`lean_kpk_broken`*); over seed-*`dk` *(z=$\mathsf{zof}(\mathtt{ek})$) binding is restored, $\leq \mathrm{CR}(H)$ under injectivity of J and* zof *(*`lean_kpk_seed_le_cr`*).*
2) Omit ct_{pq} $\Rightarrow$ K-CT survives. *The retained shared secret ss=$J(z, ct_{\mathrm{pq}})$ transitively binds ct_{pq}, so $\mathsf{Adv}^{\textit{MAL-BIND-K-CT}} \leq \mathrm{CR}(H)$ under injectivity of J (*`omit_ct_kct_le_cr`*).*
3) Omit context from the local wrapper $\Rightarrow$ its K-CTX game fails structurally. *A concrete adversary wins the self-defined context-wrapper collision game with probability* 1 *for* any `dk` *format (*`omit_ctx_kctx_broken`*). This is not a CDM result.*
4) Full combiner. *Absorbing all three gives the standard MAL-BIND-K-$\{$CT,PK$\}$ bounds and, separately, the local K-CTX wrapper bound, each $\leq \mathrm{CR}(H)$ with* no J/zof *assumption (*`full_k{pk,ct,ctx}_le_cr`*).*
5) X-Wing field shape and a separate wrapper experiment. *The native X-Wing* combiner shape *omits ct_{pq} and pk_{pq}. It loses standard K-PK* over the expanded-`dk` serialization *while keeping K-CT (*`xwing_kpk_broken`*,* `xwing_kct_le_cr`*). The K-PK break vanishes over the seed-*`dk` *serialization that deployed X-Wing mandates (cf. item 1). Native X-Wing has no context parameter and therefore cannot be assigned our K-CTX property. Separately,* `xwing_kctx_broken` *embeds the same lean field list in our context-parameterized wrapper and then ignores that wrapper input; only this hypothetical extension fails the local game.*

The two safety-of-omission *verdicts—item 2 and the seed-*`dk` *half of item 1—hold under the idealization that the implicit-rejection function J (FIPS-203's J=$\mathsf{SHAKE}(z \,\|\, H(ct))$) is* injective*; under a realistic collision-resistant-only J they degrade to an additional* CR *term over J's inner hash rather than holding outright. The breaks (the* $\Pr = 1$ *adversaries) and the full combiner (item 4) need* no *assumption on J.*

TABLE IV
THE MACHINE-CHECKED SEPARATION MAP (THEOREM 1): DELETE ONE FIELD FROM CONTEXTBOUND AND ASK WHICH BINDING NOTION SURVIVES. EVERY CELL IS AN EASYCRYPT LEMMA; J IS THE MODELED IMPLICIT-REJECTION FUNCTION.

omitted field	notion at risk	verdict
pk_{pq}	standard K-PK	**broken** / expanded-`dk` ($\Pr = 1$); safe / seed-`dk`
ct_{pq}	standard K-CT	**safe** $\leq \mathrm{CR}(H)$ (ss binds ct; needs J-inj.)
context	local K-CTX wrapper	**broken** structurally ($\Pr = 1$)
none (full)	standard CT/PK + local CTX	safe $\leq \mathrm{CR}(H)$, no J/zof
X-Wing lean fields	standard CT/PK	loses K-PK only for expanded-`dk`; keeps K-CT
ctx ignored by hypothetical wrapper	local K-CTX wrapper	**broken** ($\Pr = 1$); native X-Wing out of scope

Reading the map: The content is the *dichotomy* between rows 2 and 3, not any cell alone. Omitting ct_{pq} is safe—under the J-injectivity idealization made precise below—because the shared secret already encodes it; omitting context from the local wrapper is fatal for that wrapper game because *nothing* else does. For this local property, context absorption is necessary *and* sufficient—necessity is admittedly the easy half of an iff (no other field mentions context), sufficiency is `full_kctx_le_cr`; what makes it a result rather than a tautology is its contrast with ct_{pq}, where the shared secret *does* provide the missing binding. Conversely ct_{pq} absorption is *redundant with the shared secret*—but only under the J-injectivity idealization; the full combiner achieves K-CT with no such assumption. Row 1 is the subtle one (Schmieg's mechanism): one might *hope* $ss = J(z, ct)$ rescues pk_{pq} as it rescues ct_{pq}, but over expanded-`dk` the adversary equalizes the stored seed z across two distinct keypairs, so a garbage ciphertext implicit-rejects to the *same* $J(z, ct)$ under $\mathtt{ek}_0 \neq \mathtt{ek}_1$ [3]; the key is *independent* of pk_{pq}, so the break is genuine—not an artifact of omission (it vanishes the moment ss depends on the key, i.e. under seed-`dk`). Every verdict is supported in two different ways: explicit probability-one countermodels demonstrate the expanded-dk K-PK loss and the local wrapper's context-omission loss, while proof-dependency regression controls confirm that the safety scripts actually use their stated injectivity and encoding lemmas. These script-mutation controls are not, by themselves, logical necessity proofs. **Moral, and the formal design rationale for ContextBound: pk_{pq} is necessary for standard K-PK over expanded-`dk`; context absorption is necessary for the local wrapper property; and ct_{pq} absorption is *redundant insurance*.** This is not a stronger notion than X-Wing on a shared axis. It is a field-resolved account of the lean shape's expanded-`dk` exposure, plus a separate experiment showing exactly what a context-parameterized wrapper must absorb.

Scope, stated precisely: (i) J, zof, H are abstract; $J(z, ct)$ is *modeled* after FIPS-203 implicit rejection, not *derived* from a mechanized Decaps, and the component shared secrets are

otherwise inert. (ii) The expanded-dk K-PK break and the local wrapper's context-omission break need *no* cryptographic assumption (concrete $\Pr = 1$ adversaries); the safety bounds are conditional on $\mathrm{CR}(H)$, and the K-CT and seed-`dk`-K-PK results additionally on J/zof injectivity—a strictly stronger idealization than the full combiner needs. (iii) The K-PK break is *not* a break of X-Wing in deployment, which mandates seed-`dk`; the map characterizes the *shape*'s exposure field by field. We complement the proofs with an executable regression oracle that exercises both the expanded-`dk` break and its seed-`dk` *negative control* (where z re-derived from the keygen seed closes the substitution) against the target-selected release-graph `mlkem-native` adapter as a CI-configured gate. The predecessor `libcrux` result and failed `fips203` run are historical; the migrated test must be reported from the live release source.

C. Multi-prover assurance

Beyond the computational EasyCrypt proof, the server-authenticated hybrid handshake is modelled in two independent symbolic provers. The Tamarin model captures the ML-KEM and X25519 key exchanges, the combiner as an opaque one-way function, and an ML-DSA server signature, with rules that let the adversary compromise either KEM or reveal the signing key; it proves executability, server authentication, and—the headline lemma—*hybrid robustness*: under an honest server identity the session key remains secret unless *both* KEMs are broken. It also proves authenticated context agreement: matching accepted sessions agree on the explicit application context. A ProVerif model establishes the analogous secrecy/authentication/context queries under a different abstraction (the classical leg as a second idealized KEM), giving an independent cross-check. The current inventories are five Tamarin lemmas and six individually checked ProVerif queries. The EasyCrypt binding proof is a *containerized pinned-source hard CI gate*: a containerized EasyCrypt r2026.06 lane re-checks `BindingViaCR.ec`, the explicit $K \neq \perp$ probability-one countermodel, and the proof-dependency regression controls; CI fails if any stops holding (the no-`admit`/`sorry` grep is an additional always-on gate). The base-image digest and EasyCrypt source revision are pinned, but the apt/opam solver and transitive package graph are not; this is not a bit-reproducible or hermetic toolchain. Tamarin and ProVerif are gated on exact lemma/query inventories and full `make prove`. Tool versions are recorded by the artifact; we do not claim every base image is immutable by digest.

VI. Production Integration and Evaluation

Evidence currentness: Every committed timing value in this section was collected on predecessor `libcrux`-era source. The research-alpha release graph now uses target-selected `mlkem-native`/`fips204`/`sha3`; the source digest changed, so these values remain historical study data and are not current ABI 2 performance evidence. A separate same-host native/portable diagnostic is reported below, but it was not a clean canonical-source-bound release capture.

A. Microbenchmarks

Table V reports a committed historical snapshot of primitive and hybrid costs (Criterion, point estimates, on an Apple-Silicon arm64 host). The data is `paper/microbench-arm64.csv`, regenerable with `cargo bench -p q-periapt-backends`. These numbers establish scale on one host, not current cross-version or cross-device performance parity. Two historical backend-specific observations matter for deployment. First, *in the predecessor libcrux/x25519-dalek backend the post-quantum leg was not the bottleneck*: ML-KEM-768 encapsulation is $11.0\,\mu$s, roughly $8\times$ cheaper than X25519 encapsulation ($89.8\,\mu$s, an ephemeral keygen plus a Diffie–Hellman), so the classical X25519-as-KEM path dominates hybrid CPU here—a corrective to the assumption that PQ is the expensive part (a different backend, e.g. the aws-lc-rs `X25519MLKEM768` row, shifts this balance). Second, *the hash-everything combiner is cheap*: In that snapshot, ContextBound costs ~ 6–$8\,\mu$s over the byte-exact X-Wing combiner CompatXWing (encapsulation 109.1 vs. $101.3\,\mu$s; decapsulation 65.9 vs. $59.2\,\mu$s; variance $\sim 1\,\mu$s). This supports a sub-$10\,\mu$s local combiner-overhead observation for that run; it does not support the stronger claim that the complete stack is never slower than X-Wing or other PQC deployments.

We therefore added one dual-estimand paired gate. The profile-non-regression estimand removes the earlier backend/key-format confound: ContextBound and CompatXWing use the same native ML-KEM-768 seed-`dk` + X25519 backend, keys, coins, 64-case deterministic ciphertext corpus, and ABBA/BAAB schedule. Raw schema v4 fixes their canonical inputs separately: ContextBound uses the fixed suite, policy version, and application context, whereas CompatXWing uses their canonical absence (empty, zero, empty). The separately named implementation-improvement estimand compares native and evidence-only portable ML-KEM-768 implementations for ContextBound encapsulation and decapsulation in the same harness process, after requiring byte-identical keys, ciphertexts, and shared-secret outputs. Both use the fixed ABBA/BAAB schedule. After a 5-s warm-up, raw schema v4 records 20,480 paired samples per applicable operation and variant. Each timed sample batches 256/1/2 calls for combine/encapsulation/decapsulation, records the unrounded total, and normalizes only after a strict iteration-map check. Consecutive 1,024-pair, corpus-balanced blocks define both the primary paired point estimate and its span-5 moving-block-bootstrap upper bound. Under the nearest-rank rule each block's p99 has 11 tail observations. Budget schema v9 preserves the v6 profile estimator and its former 256-pair regression guard, requiring every published profile limit to pass at both scales. Separate 64/256/256-pair windows test temporal stability without changing the 5% CV limit.

A controlled Apple-Silicon proof is accepted only if it passes the nine profile-non-regression and six implementation-improvement *per-metric* one-sided 95% bootstrap budgets (not a simultaneous 95% family guarantee; span-5 coverage under autocorrelation has not been independently calibrated). The profile bounds retain p50/p95/p99 ratios of 1.10/1.15/1.20 and a $15\,\mu$s p95 absolute delta for encapsulation/decapsulation,

plus a $10\,\mu$s combiner p95 delta. For both native/portable ContextBound product operations, the implementation bounds require p50 and p95 upper ratios at most 0.95 and p99 at most 1.0. Proof schema v7 binds the raw samples, final harness binary, evidence-only portable archive and source, verifier-fixed repository budget, commit, canonical source-input tree, selected Rust target/toolchain, and the macOS SDK path, version, and settings digest. This is not yet a hermetic build-input closure. Raw, budget, and proof semantics share their digest snapshots; four boundary power/thermal observations are gated. The producer fixes Cargo, Rustc, Xcode Clang, and Xcode `ar` executable paths, hashes, and version output where available; it rejects repository/ancestor/user Cargo configuration and caller compiler/wrapper/loader controls, fixes system-tool lookup, and builds offline in a fresh private target. It still trusts the user-writable Cargo registry, Rust sysroot/driver, OS tools/libraries, same-UID host, and collector source-to-binary honesty; the verifier does not independently rebuild the binary. Clean provenance checks HEAD/index/actual bytes under a fixed Git environment. The results-manifest digest is pinned across one verifier run; the manifest root and historical camera transcript remain outside that non-self-referential digest with distinct Git/hash provenance, and exact run values are retained in the machine-readable artifact. A proof counts as current only when its source digest matches the live canonical tree and the host satisfies the controlled-environment contract; the manifest carries its path/hash/schema/source/pass summary, while the required domain verifier checks the actual proof, artifacts, and freshness. This source text cannot self-promote a stale run. A passing snapshot closes only a single-host matched-core diagnostic for those bytes, not device energy, rustls end-to-end, or optimized production-X-Wing parity. The migrated harness identifies `mlkem-native` v1.2.0 + `sha3` 0.10.9 + x25519-dalek 2.0.1; every previously recorded paired proof has a different source/backend identity and must be recollected.

A local dirty native/portable diagnostic indicated material ML-KEM-768 primitive and product-path improvement. It has no checked-in canonical-source/toolchain-bound raw and analysis bundle and therefore is not formal release evidence, an optimized production-X-Wing comparison, or a device/protocol performance claim. Exact quantitative results require a fresh raw-schema-v4/proof-schema-v7 run under budget schema v9, selected by the current results manifest.

B. Handshake latency under emulated networks

We expose ContextBound (and CompatXWing) as a TLS 1.3 key-exchange group via a rustls [38] `CryptoProvider`, using private-use group codes; a real loopback TLS 1.3 handshake completes over this provider, with the server authenticated by a *standard TLS certificate* (a self-signed certificate in the loopback measurement, via the rustls cipher-suite/signature machinery—*not* an ML-DSA signature). Figure 5 shows the key-exchange flow: the client offers a combined keyshare, the server encapsulates to both components and derives the session key with the combiner, and the client decapsulates and re-derives the same key—which stays secret unless *both* component KEMs are broken. We evaluate handshake *time-to-session* over a real loopback socket shaped by `tc netem`,

TABLE V
HISTORICAL PREDECESSOR-SOURCE PRIMITIVE AND HYBRID COST (μS; CRITERION 60-SAMPLE POINT ESTIMATES, ARM64 HOST) AND KEY/CIPHERTEXT SIZES (BYTES). A SINGLE-HOST SUPPORTING MICROBENCH (RUN-TO-RUN VARIANCE $\sim 1\,\mu$S); THE BARE-METAL TIME-TO-SESSION OF TABLE VI IS THE PRIMARY MEASUREMENT.

	time (μs)			**size (B)**		
Scheme	keygen	encaps	decaps	ek	dk	ct
ML-KEM-512	6.5	6.1	6.8	800	1632	768
ML-KEM-768	11.2	11.0	11.9	1184	2400	1088
ML-KEM-1024	18.2	16.2	17.0	1568	3168	1568
X25519	43.5	89.8	44.4	32	32	32
Hybrid, ContextBound	—	109.1	65.9	1216	—	1120
Hybrid, CompatXWing	—	101.3	59.2	1216	—	1120

comparing a classical X25519 baseline against the two PQ/T profiles. Because the metric is *TCP connect plus a full TLS 1.3 handshake*, the floor is about two RTTs before any local CPU cost. The *ML-DSA*-authenticated hybrid handshake—whose robustness our symbolic models establish (Section V)—is a separate HPKE-shaped TCP demo, *not* the TLS 1.3 wire format, and is not used for the measurements here.

Figure 6 shows the qualitative picture—the curves coincide because latency is RTT-dominated—and Table VI gives the quantitative camera-ready measurement on quiesced bare-metal hardware (an 8-core AMD Ryzen 7 7700, Ubuntu 24.04, kernel 6.8, the benchmark pinned to two selected cores on a quiesced host under the `performance` governor; medians over 20 repetitions). At RTT ≈ 0 the per-group medians are tight to $\pm$2–3 μs and the ordering is resolved at the median (ContextBound $>$ CompatXWing in 18 of 20 reps): classical X25519 238 μs, the IANA-standard `X25519MLKEM768` 263 μs, CompatXWing 318 μs, ContextBound 326 μs. The p99 tracks the p50 (Table VI: 252.7 / 317.1 / 377.5 / 385.9 μs), so the ContextBound$-$CompatXWing gap is $+8.4\,\mu$s at the p99—the combiner cost does not grow in the tail. The reproducible local overhead of this Q-Periapt rustls path over classical X25519 is thus $\approx$ 88 μs, and—critically—**ContextBound costs $+8.1\,\mu$s over CompatXWing**, the price of explicit hash-everything transcript binding, in close agreement with the $+6.2\,\mu$s combiner cost measured in isolation (Table V). This quiesced measurement supersedes our earlier virtualized-host run, whose $2\times$ run-to-run swings exceeded the few-μs combiner difference; on bare metal it resolves cleanly. The committed raw capture predates the hardened archive/tool/binary-bound harness and is therefore historical data, not a current-tree proof; a new bare-metal capture must pass the shipped raw-bundle verifier before these values can be promoted to final artifact evidence. Even then, the bundle is producer-origin integrity evidence: without an external challenge nonce and separately trusted TPM/signing identity, its hashes do not constitute independent hardware attestation or prevent wholesale replay.

As RTT grows this fixed cost vanishes into the network: at RTT $=$ 20 ms all four groups sit at 41.1 ms within a 90 μs spread, and at RTT $=$ 50 ms within 15 μs of one another—the combiner difference is two to three orders of magnitude below the link latency. The hybrid's $+2.27$ KB key-

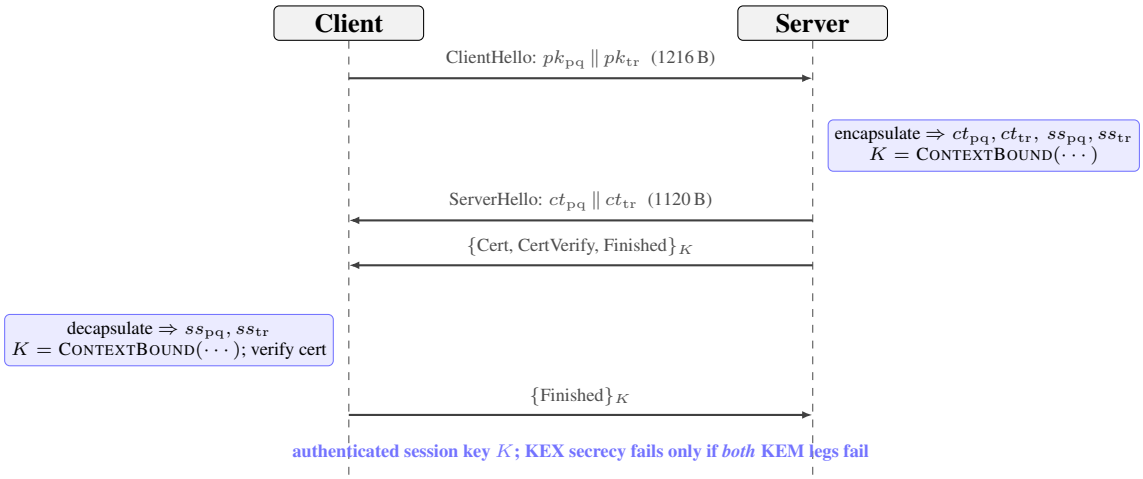

Fig. 5. PQ/T hybrid TLS 1.3 *key exchange* over the rustls provider (private-use group), with standard TLS server-certificate authentication (a self-signed certificate in the loopback measurement). Both peers derive the same session key K with the ContextBound combiner; under the standard TLS server-authentication assumption, K's secrecy fails only if *both* component KEMs are broken. ML-DSA-based authentication is the separate HPKE-shaped demo of Section V, not this measured TLS path.

TABLE VI
HISTORICAL PREDECESSOR-SOURCE TIME-TO-SESSION ON QUIESCED BARE-METAL HARDWARE (AMD RYZEN 7 7700, BENCHMARK PINNED TO TWO SELECTED CORES, `PERFORMANCE` GOVERNOR; MEDIAN OF THE PER-REP P50 AND P99 OVER 20 REPETITIONS, RTT-0 COLUMN). AT RTT 0 THE PER-GROUP ORDER RESOLVES AND CONTEXTBOUND IS $+8.1\,\mu$S OVER COMPATXWING ($+8.4\,\mu$S AT THE P99). THE TAIL CLAIM IS MADE AT RTT 0, WHERE THE COMBINER COST IS RESOLVABLE; AT RTT ≥ 20 MS THE PER-GROUP SPREAD IS $\leq 90\,\mu$S AGAINST A ≥ 41 MS LINK, SO ANY TAIL THERE REFLECTS NETWORK JITTER RATHER THAN THE COMBINER (MEDIANS SHOWN, 6/4 REPS). THE HISTORICAL RAW TRANSCRIPT IS IN `PAPER/CAMERA-READY-RESULTS.TXT`; IT LACKS THE SOURCE ARCHIVE, BINARIES, LOGS, QDISC SNAPSHOTS, AND BUNDLE MANIFEST REQUIRED BY THE CURRENT HARNESS/VERIFIER, WHICH MUST BE RERUN BEFORE PROMOTION.

	RTT 0			
key-exchange group	p50	p99	RTT 20 ms	RTT 50 ms
X25519 (classical)	238.3 μs	252.7 μs	41.06 ms	101.16 ms
X25519MLKEM768 (std.)	263.5 μs	317.1 μs	41.15 ms	101.17 ms
CompatXWing	317.8 μs	377.5 μs	41.14 ms	101.17 ms
ContextBound	325.9 μs	385.9 μs	41.13 ms	101.17 ms

exchange increment (one ML-KEM-768 keyshare each way; the authentication certificate/signature bytes are unchanged from the classical baseline) fits within the existing TLS flights, adds no round-trip, and is byte-identical in wire budget to the IANA standard. The two hybrids differ in *two* independent ways, which the table separates: the combiner (ContextBound vs. concatenation), worth $+8.1\,\mu$s (the `bound-compat` delta, same ML-KEM implementation), and the ML-KEM implementation itself ($+62\,\mu$s between the `standard` row, on aws-lc-rs, and the historical `bound` row, on libcrux/ring)—an implementation-not-combiner cost we do not optimize; isolating it would be the right job for an aws-lc-rs-backed ContextBound implementation, which we do not build—so we do not claim the $62\,\mu$s eliminated, only that it is backend-related. The binding upgrade ContextBound provides is the $+8.1\,\mu$s, not the 62.

Under jitter and loss. Adding 3 ms of jitter leaves the picture unchanged (all three groups ≈ 41 ms median, ≈ 50 ms p99.9). Under 1–3% packet loss the median is likewise unaffected—loss is rare relative to a handshake—but the *tail* jumps to ~ 1 s, dominated by a TCP retransmission timeout per lost packet rather than by crypto or wire size; this RTO-driven tail falls on the classical and PQ/T handshakes alike, and at our sample size the loss-event noise precludes a precise per-group tail comparison. The takeaway is that the hybrid's extra keyshare does not *systematically* worsen loss resilience: the median is byte-insensitive and the tail is governed by transport, not by the combiner.

VII. DISCUSSION

When to use which profile: `CompatXWing` is the construction-compatible control: it is byte-exact against the official X-Wing vectors, but independent endpoint/HPKE interoperability remains untested. `ContextBound` is the default for first-party deployments and is preferable whenever (i) the application has a meaningful context (a protocol/role/version label, a channel binding) that the session key should commit to; or (ii) the deployment cannot guarantee that its component KEM will only ever be instantiated in the seed-dk form—in which case binding pk_{pq} and ct_{pq} directly removes the latent serialization dependence of Section V. In the historical predecessor single-host snapshot, that robustness adds under $10\,\mu$s at the combiner layer; end-to-end invisibility and parity remain deployment-specific claims requiring current paired measurements.

The assurance philosophy: Our position is deliberately modest about cryptographic novelty and deliberately ambitious about *dependability*: a hybrid is trustworthy only to the extent that its weakest realization is. The recurring failures of the last two years—KyberSlash, PQXDH, the ML-KEM malicious-key binding gaps—were not failures of the underlying hard problems but of *realizations*: a compiled branch, a serialization format, a missing transcript field. The contribution of this paper is a discipline that makes those realization-level properties *checkable and reproducible*: a binding result tied to one

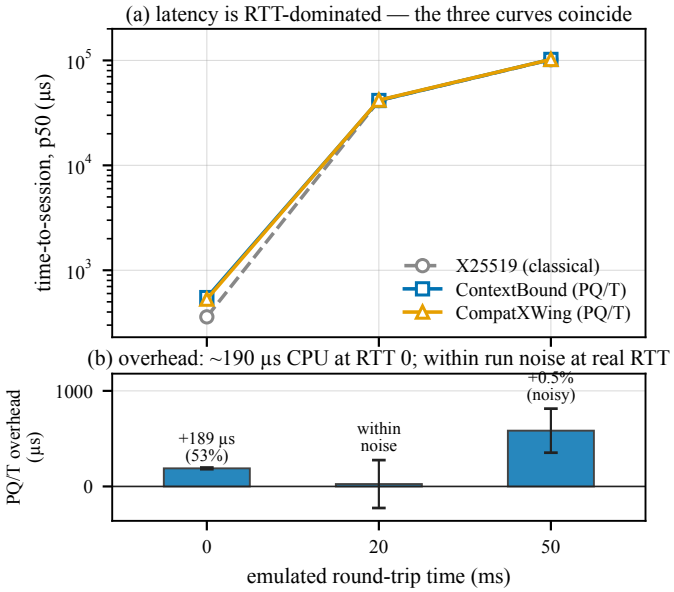

Fig. 6. Historical predecessor-source time-to-session under real `tc netem` on the *virtualized* host (mean of two repetitions); the quiesced bare-metal numbers of Table VI supersede it quantitatively, and this figure is retained for the qualitative shape. (a) latency is RTT-dominated—the three curves coincide. (b) the overhead is the fixed local KEX-path overhead at RTT 0 ($\sim$190 μs on this virtualized host; $\approx$88 μs on bare metal, Table VI); at realistic RTT it is within run-to-run noise (error bars span the two repetitions and cross zero at RTT 20 ms).

implementation through a byte-conformance matrix, a constant-time probe that discriminates clean from leaky, and a regression oracle for the key-format coupling. We expect the individual instruments to be reused independently of ContextBound.

Generality: Nothing in the approach is specific to ML-KEM + X25519. The combiner and its proof are generic over the component KEMs; the publishable trait-injected backends include the full ML-KEM and ML-DSA families. The former PQClean-HQC adapter has been removed. A separate non-publishable RustCrypto HQC-v5/FIPS-207-draft candidate shadow tests only its declared research boundary and owns no product-suite code or ABI. Although its upstream describes it as tracking an IPD, as of 2026-07-12 NIST still labels FIPS 207 as coming soon and no official IPD is retrievable; freezing and testing the eventual official text is a promotion gate. As new PQ primitives are standardized, the same proof-to-byte discipline can be applied after an explicit security/API and release review.

VIII. Related Work

KEM binding and committing security: The systematic study of KEM binding is recent. CDM [2] introduced the X-BIND-P-Q lattice, the HON/LEAK/MAL adversary hierarchy, and its monotonicity, and showed where standard KEMs sit; Schmieg [3] then proved that ML-KEM in its FIPS-203 *expanded* decapsulation-key form is neither MAL-BIND-K-CT nor MAL-BIND-K-PK, with seed-form keys as the fix, and [26] analyzes implicitly-rejecting KEMs more broadly. For *combined* KEMs, Chempat [4] establishes the dichotomy we rely on—a hash-everything combiner is binding from collision-resistance alone, while a lean combiner inherits the binding of whatever component it omits; ContextBound is an instance of the former, which is exactly why we attribute the binding *fact* rather than claim it. At the symmetric layer, the same idea appears as context-committing AEAD (CMT) [7]; our self-defined context-wrapper projection is a syntactic KEM-output-layer analogue, not a CDM axis. Our separation map (Theorem 1) sits one level finer than each of these: where Chempat states a *binary* dichotomy (the lean shape inherits the omitted component's binding) and Schmieg fixes a *single* KEM's format-dependence, the map is a *field-resolved* statement—per binding notion and per `dk`-format, exactly which combiner input is load-bearing and which is made redundant by the shared secret. It shows that the public key is load-bearing over expanded-`dk`, while context is load-bearing only for the explicit wrapper property and the PQ ciphertext can be redundant under the stated idealization. We make no priority claim for the concept of context absorption. The necessity claims rest on explicit probability-one countermodels; the proof-dependency controls (Section B-A) only guard against accidental weakening of the proof scripts. Our contribution is thus twofold: this separation map, and the conjunction of Table I—a mechanized, assumption-minimal binding result tied by explicit conformance evidence to one implementation across the reported substrate.

Side-channel assurance: KyberSlash [6] showed that secret-dependent timings in widely-used Kyber/ML-KEM implementations were exploitable, underscoring that source-level constant-time arguments do not by themselves settle the question for shipped binaries. The dudect/ctgrind discipline of marking secrets and checking for secret-dependent control flow [34], [36], realized over Valgrind/Memcheck [35], is the basis of our binary-CT gate. We build on, rather than re-do, the literature on HACL* [27] and predecessor libcrux ML-KEM verification [28]. That source-level assurance and our former dynamic zero result are historical and do not transfer to `mlkem-native`. The target-selected gate retains the synthetic planted leak, but its genuine-secret path needs fresh x86-64-portable/aarch64-native captures. Older-source runs also discriminated the former path from the now-retired timing-leaky PQClean-HQC path; those results are not current-source evidence or a blanket verification of any dependency.

Hybrids in deployment: Hybrid key exchange is being standardized and shipped: the IRTF combiner draft's `UniversalCombiner` and C2PRI profiles [17], the IETF hybrid framework [16] and the `X25519MLKEM768` group [19] for TLS 1.3 [15], [14], the X-Wing KEM [5], Signal's PQXDH (formally analyzed in [20]) followed by its SPQR/ML-KEM-Braid Triple Ratchet [22], [23], [25] and Sesame multi-device session management [24], and Apple's PQ3 [21]. X-Wing is presently an individual, intended-Informational Internet-Draft rather than an IETF Standards-Track RFC. These are the baselines we position against; our `CompatXWing` profile reproduces the official X-Wing KEM vectors, while our rustls integration targets the same TLS 1.3 deployment path. We do not claim interoperability with an independent X-Wing or HPKE endpoint. The current IRTF hybrid-KEM draft-12 labels its Section 6.4.2 binding arguments informal LEAK-BIND sketches, defers rigorous proofs, and does not prove the potential common-seed MAL strengthening [17]. Thus our

defensible delta is machine-checked, field-resolved standard MAL-BIND-K-CT/K-PK reductions, a separately scoped local K-CTX wrapper reduction, and executable countermodels—not the already-published current-draft idea of absorbing all fields.

Formal verification of PQ cryptography: formosa-mlkem [29] delivers a verified ML-KEM *implementation* (Jasmin/EasyCrypt) with IND-CCA and constant-time guarantees—an axis (specification-to-binary) we explicitly do not hold (Table I). The SandboxAQ EasyCrypt-KEMs library [30] mechanizes a proof that ML-KEM satisfies LEAK-BIND-K-PK (in the ROM). Our EasyCrypt [31] development is complementary: it targets the *hybrid combiner's* binding (the commitment-style MAL Figure-6 instantiation), cross-checked by independent symbolic models in Tamarin [32] and ProVerif [33]; the EasyCrypt proof is hard-gated in a pinned-source container in CI, while the symbolic models are hard-gated on both lemma/query presence and full prover execution. Signal reports a stronger implementation-linkage baseline for its stateful ratchet crate: ProVerif state-machine models plus hax-to-F* checks of core Rust pre/postconditions and panic freedom on every CI run [25]. Our proof-to-byte chain is conformance and provenance evidence, not a mechanized refinement result, and we make no superiority claim on that axis.

IX. Limitations

The mechanized binding result is over an *abstract* Decaps: it is exactly the "zero KEM assumption" that makes the result general, but it carries no FIPS-203 linkage, the shared-secret fields are inert in the argument, and CR(SHA3) and IND-CCA2 remain assumptions argued respectively in the model and on paper; there is no specification-to-implementation linkage proof. Binary-level constant-time tooling is configured only on x86-64 and aarch64, and the target-selected `mlkem-native` paths still need fresh x86-64-portable and aarch64-native runs; the failed `fips203` run is historical, and riscv64 and wasm32 have no current binary-CT or inherited source-CT claim. ACVP byte-identity is conformance, not CMVP certification. Upstream HOL-Light evidence covers only selected upstream AArch64 assembly source/object routines, not downstream reassembly or the full ABI. The suite is research-grade and has not had an independent third-party audit. The historical netem latency numbers (Table VI) are from a quiesced bare-metal host; the earlier virtualized-host run (Fig. 6) and the jitter/loss study (Table IX) are retained as supporting context and labeled as such. The matched-backend host gate accepts only a controlled proof whose digest matches the live canonical source tree; its machine-readable manifest carries the current state. Snapshot consistency does not turn a dirty mutable manifest into a trust root or atomically freeze a writable worktree; release provenance still requires a clean committed, signed, or transparency-backed root. Even a current one-host pass would not establish performance parity with current X-Wing, TLS, Signal's combined PQXDH/Triple-Ratchet stack, PQ3, or other PQC implementations; device energy, optimized backend, end-to-end, and stable clean-baseline evidence remain open. Current release attestation additionally requires a clean, source-bound Apple device matrix; a dirty-tree diagnostic or a prior device result is not such an attestation. Removing the PQClean dependency graph and tombstoning suite code 3 invalidated the pre-change Apple/performance proofs. A later predecessor snapshot recorded a clean-tree schema-v3 Apple matrix and controlled-host matched-backend proof results, but the release-graph backend migration changed the source digest again; all recorded package, device, performance, and binary-CT evidence is therefore historical and must be regenerated. The migration removed the `libcrux`/hax `proc-macro-error2` edge, and `cargo audit --deny warnings` now passes without an ignore. This dependency scan is not an independent cryptographic or ABI audit. ABI 2 is the stable-version source/Rust-crate line; package readiness is not registry publication. Its stable binary targets are immutable, attestation-bound GitHub releases: an Apple Developer ID-signed XCFramework revision and a platform distribution carrying the Android AAR with API 35 / 16 KiB-page emulator runtime evidence and Linux x86_64/aarch64 SDK archives. The unsigned Windows SDK remains an unsupported CI diagnostic outside the formal release. Production promotion still requires such review, fresh release-bound two-ISA CT, same-source device/performance evidence, clean signed or transparency-backed source provenance and registry publication. The rustls integration uses a fixed protocol-domain context and an already parsed, unauthenticated policy input; it therefore neither carries arbitrary per-session K-CTX nor belongs to the signed-policy authenticity claim. The C ABI assumes a trusted local caller for metadata and the forgeable decision descriptor, despite removing raw/deterministic cryptographic exports, and its exact-nine claim applies to the dynamic `q_periapt_*` export table. Static archives constrain that public namespace but retain unsupported hidden `qpn_*` bridge link symbols; hidden visibility is not same-process access control. The former known-leaky PQClean-HQC adapter is removed rather than shipped as a fallback. Its numeric suite code is a permanent tombstone; the standalone HQC draft-candidate shadow has no ABI or product identity. Finally, policy signatures authenticate configuration bytes; decision descriptors, WASM raw inputs, and same-process opaque handles do not turn a hostile in-process caller into an authorization boundary. The artifact is not an asynchronous session protocol: it has no account/device directory, one-time prekeys, persistent ratchet, multi-device convergence, recovery, or key transparency. Signal and PQ3 remain ahead on those lifecycle and deployment axes. A separate, non-normative Q-Periapt Continuity research model now explores canonical typed lifecycle-context admission, a strict model-only two-leg prekey-selection record, exact version-and-digest repository advances, and no-op-anchor rejection, but deliberately has no context-advance API, credential/manifest/prekey service, production wire, ratchet, provider-policy authorization, real persistence adapter, or implementation-refinement claim. It is future work and is not evidence for any claim in this paper.

X. Conclusion

A PQ/T hybrid is only as safe as its weakest realization. We narrowed that gap with an assumption-minimal, machine-checked binding result and harnesses that tie it by conformance

to one implementation across a heterogeneous deployment surface, guarded by reproducible gates (configured in CI)—a source→binary constant-time discriminator, a binding key-format coupling demonstration, and multi-prover symbolic models—plus an atomic, authenticated policy-to-byte path. The artifact-specific contribution is the conjunction, not an exhaustive priority claim over any single fact; predecessor-source measurements remain historical, and the migrated backend requires fresh CT, conformance, package, device, and performance execution. A machine-readable ledger keeps proved, tested, device-bound, and pending claims separate. Future pairwise-protocol work will first build component-conformant PQXDH and Triple-Ratchet references plus a separately specified Sesame-compatible manager integration. It will then evaluate authenticated policy/context continuity, prekey accountability, crash/rollback invariants, schedule-relative PQ-healing debt, native Apple PQ providers, metadata privacy, and proof-to-state-to-byte evidence under physical-device wire/latency/energy budgets; it remains deliberately outside the present contribution and is not messaging-product parity.

APPENDIX A
ARTIFACT AVAILABILITY

The complete suite—Rust core and backends, the deterministic conformance surfaces, the native ABI 2 product adapters, the EasyCrypt/Tamarin/ProVerif developments, the constant-time and netem harnesses, and the figure-generation scripts—is prepared for a history-free anonymized export through the editor (public URL withheld for double-blind review). The development repository and raw local device directories are not themselves suitable review artifacts: the export must remove identity and publicly linkable history, then regenerate all source-bound proofs from the anonymous root snapshot. The resulting snapshot contains the committed historical measurements, inputs, and regeneration protocols; no committed predecessor result is current ABI 2 evidence, and exact timing reproduction remains host- and toolchain-dependent.

Independently of the review export, the stable-version binaries use receipt-gated immutable GitHub releases with `prerelease=false`, bound to annotated tags, per-asset SHA-256 manifests, and GitHub immutable-release and build-provenance attestations: an Apple Developer ID-signed static XCFramework revision, and one platform distribution containing the four-ABI Android AAR with an API 35 / 16 KiB-page emulator runtime-evidence bundle and GNU/Linux x86_64 and aarch64 SDK archives. The unsigned Windows SDK remains an unsupported CI diagnostic and is excluded from the stable manifest, assets, attestation, and receipt. Machine-checked publication receipts are recorded in the repository evidence manifest and enforced by a fail-closed release contract. Stable publication is not a production-readiness claim: no unverified registry (crates.io, Maven Central, deb/rpm/MSIX) publication, physical-Android coverage, independent certification, or FIPS validation is claimed.

APPENDIX B
THE MACHINE-CHECKED KERNEL IN DETAIL

Table VII inventories the EasyCrypt development. Of the named results, all are fully discharged (no `admit`/`conjecture`). For the full ContextBound binding theorem the only structural encoding assumptions are the two elementary `be8_*` facts about the 8-byte length field plus the modeling of H as collision-resistant; the separation-map rows add the explicitly stated existence/injectivity assumptions (`ek_neq`, `lctx_neq`, `zof_inj`, `jrej_inj`) shown in their row statements. We guard the proof scripts with *proof-dependency regression controls*: for each binding theorem we mechanically remove a selected closing hint and re-run the checker, confirming that the current script then *fails* (`cannot prove goal`). Removing `encode_inj` breaks every binding theorem; deleting the disagreement lemma (`ct_neq_fields_neq`, etc.) breaks the corresponding axis. These mutations show that the checked scripts depend on their intended lemmas; they do not prove logical necessity. Separately, an explicit probability-one countermodel for the game with the $K \neq \bot$ requirement removed shows that two mutually rejecting executions can otherwise "win" without inducing a hash collision. That countermodel, rather than a failed tactic, establishes why the Figure-6 conjunct is required.

A. Proof structure of the separation map

All parts of Theorem 1 are discharged by two idiomatic skeletons over a single `lexec` execution type, so the map is genuinely *one* model rather than a family of bespoke arguments. *Safety* bounds (`full_*`, `omit_ct_kct_le_cr`, `lean_kpk_seed_le_cr`, `xwing_kct_le_cr`) are `byequiv` reductions to the collision game: a binding winner—two executions whose combiner keys collide while the observable differs—is mapped to the pair of *encodings* of their field lists, which are distinct yet hash-equal, i.e. an H-collision. The list-difference step is the only non-boilerplate part, and it is exactly where each result's assumption enters. When the omitted observable still sits in the field list (the `full_*` rows), plain injectivity of the encoding suffices. When it does *not*—the crux of `omit_ct_kct_le_cr`, where ct_{pq} is absent—a differing ciphertext must instead force a differing *shared secret* $J(z, ct_{\mathrm{pq}})$, which it does *only* under injectivity of J (`jrej_inj`). That single implication is the formal content of "the shared secret transitively binds the ciphertext," and isolating it is what keeps the K-CT result sound rather than circular. *Breaks* (`lean_kpk_broken`, `omit_ctx_kctx_broken`, `xwing_k{pk,ctx}_broken`) are `byphoare` arguments exhibiting a concrete deterministic adversary that outputs two executions agreeing on every *absorbed* field but differing on the omitted observable; the probability-one win is then a structural computation whose sole hypothesis is that two distinct observables exist (`ek_neq`, `lctx_neq`).

Proof-dependency regression controls and explicit countermodels. A passing machine proof certifies only that the conclusion follows from the stated hypotheses; it does *not* certify that the hypotheses are needed, and a vacuous or over-strong premise is the classic way a mechanized result

TABLE VII
EASYCRYPT LEMMA INVENTORY (BINDINGVIACR.EC). ALL PROVED; FULL-COMBINER BINDING USES BE8_* AND $\mathrm{CR}(H)$; THE SEPARATION-MAP ROWS ADDITIONALLY USE THE ELEMENTARY ASSUMPTIONS EK_NEQ/LCTX_NEQ/ZOF_INJ/JREJ_INJ STATED IN THEIR ROW STATEMENTS.

Lemma	Statement (informal)
lp_cat_inj	length-prefixed fields are self-delimiting
encode_inj	the field encoding is injective
bind_le_cr	generic transcript-collision $\leq \mathrm{CR}(H)$
bind_le_cr_k*	instantiated projections (CT/PK/CTX)
malbind_k*_le_cr	standard CT/PK plus local CTX wrapper, implicit rejection $\leq \mathrm{CR}(H)$
malbind_*_xrej_le_cr	CDM Fig. 6 for CT/PK; same rejection skeleton for K-CTX, $K \neq \perp$
Separation map (Theorem 1): single-field deletions	
lean_kpk_broken	omit pk_{pq}, expanded-dk: K-PK adversary $\Pr = 1$
lean_kpk_seed_le_cr	omit pk_{pq}, seed-dk: K-PK $\leq \mathrm{CR}(H)$ (J/seed inj.)
omit_ct_kct_le_cr	omit ct_{pq}: K-CT $\leq \mathrm{CR}(H)$ (ss binds ct, J-inj.)
omit_ctx_kctx_broken	omit context: local-wrapper adversary $\Pr = 1$ (any dk)
full_k*_le_cr	full shape: standard K-{PK,CT} plus extended K-CTX $\leq \mathrm{CR}(H)$
xwing_kpk_broken	X-Wing lean fields: loses K-PK for expanded-dk, $\Pr = 1$
xwing_kctx_broken	hypothetical context wrapper around lean fields ignores ctx: local game $\Pr = 1$; native X-Wing out of scope
xwing_kct_le_cr	X-Wing: keeps K-CT $\leq \mathrm{CR}(H)$

misleads. For each relevant script we therefore re-ran the checker with a selected proof hint removed from the smt invocation and confirmed the script *fails* (reproducible via negative-controls.sh, whose log is in the artifact): the K-PK break script fails without two distinct keys, the K-CTX break without two distinct contexts, the K-CT safety without J-injectivity, the seed-dk K-PK safety without J/zof injectivity, and every reduction without the proved encode_inj. These controls protect against accidental script weakening but do not turn sufficient conditions into logical necessity. Necessity for public-key and context absorption is instead supported by the explicit probability-one adversaries, and the need for the $\mathsf{K} \neq \perp$ conjunct by its explicit probability-one countermodel. The whole development is admit-free and rests on six elementary axioms (be8_size, be8_inj, ek_neq, lctx_neq, zof_inj, jrej_inj); H's collision resistance is the sole *cryptographic* assumption, and the injective encoding is a proved lemma, not an axiom.

Assumption ladder. The map's verdicts do not all sit at the same assumption level, and the differences are themselves informative. The two breaks and the full-combiner safety bounds need *no* idealization of J: the breaks are structural probability-one adversaries, and the full combiner reduces to $\mathrm{CR}(H)$ alone because it absorbs every field directly. The two

TABLE VIII
MICROBENCHMARKS, μS [95% CI]. ARM64 HOST, CRITERION.

Scheme	keygen	encaps	decaps
ML-KEM-512	6.5 [6.4,6.6]	6.1 [6.0,6.3]	6.8 [6.8,6.9]
ML-KEM-768	11.2 [11.2,11.3]	11.0 [10.9,11.1]	11.9 [11.8,12.0]
ML-KEM-1024	18.2 [17.9,18.5]	16.2 [15.8,16.7]	17.0 [16.8,17.2]
X25519	43.5 [43.2,43.9]	89.8 [89.2,90.5]	44.4 [44.1,44.7]
Hybrid, ContextBound	—	109.1 [108.4,109.9]	65.9 [65.5,66.4]
Hybrid, CompatXWing	—	101.3 [100.7,101.9]	59.2 [58.8,59.5]

TABLE IX
TIME-TO-SESSION UNDER JITTER/LOSS, P50/P99 (μS), ON THE VIRTUALIZED HOST (SUPPORTING CONTEXT; THE PRIMARY PER-GROUP LATENCY IS THE BARE-METAL TABLE VI). THE P99 UNDER LOSS IS RTO-DOMINATED AND STATISTICALLY INDISTINGUISHABLE ACROSS GROUPS.

netem (RTT 20 ms +)	X25519	ContextBound	CompatXWing
+3 ms jitter	41.5k / 49.9k	41.6k / 50.1k	41.2k / 49.5k
+1% loss	41.1k / ∼1.08M	41.3k / (noisy)	41.4k / ∼1.09M
+3% loss	41.2k / ∼1.09M	41.6k / ∼1.09M	41.5k / ∼1.10M

conditional verdicts—K-CT safety after omitting ct_{pq}, and seed-dk K-PK safety—are precisely the ones that lean on the shared secret to do binding work the encoding no longer does, and they pay for it with the J/zof injectivity idealizations. So the assumption a reader must grant scales with how much the combiner offloads onto the component KEM's internals: zero additional component-KEM binding/injectivity assumptions for ContextBound's direct absorption beyond $\mathrm{CR}(H)$, and more for every shortcut. That monotonicity is the quantitative form of the paper's thesis that absorbing the full transcript is the assumption-minimal choice.

APPENDIX C
EXTENDED EVALUATION

Table VIII gives the microbenchmarks of Table V with Criterion's 95% confidence intervals (arm64 host). Table IX gives the jitter/loss data of Section VI: the median is loss-insensitive, while the tail is governed by a per-loss TCP retransmission timeout and is, at 150 samples, statistically indistinguishable across the three groups—hence we report it as "RTO-dominated, transport-bound" rather than as a per-group claim. Current-source local footprint (platform/toolchain-dependent, paper/footprint.csv): on Darwin 27.0.0 arm64 with Rust 1.96.1, wasm-pack 0.15.0, and Homebrew LLVM Clang 22.1.8, the stripped native ABI 2 library is 667.8 KiB, lean WebAssembly is 97.7 KiB, and signed-policy WebAssembly is 332.3 KiB. These are local diagnostics rather than signed provenance or cross-platform package-size evidence; regenerate each claimed platform/toolchain from the selected source.

APPENDIX D
REPRODUCTION

The artifact ships a layered guide (ARTIFACT.md) in three tiers by cost and dependency weight. *Tier 1 (~10 min, Rust only):* cargo test --workspace (KATs, NIST ACVP, cross-crate differential, proptests, host-side

FFI/WASM vectors), `cargo fmt --all --check`, and the no-`admit`/`sorry` proof grep. *Tier 2 (~1 h, the CI gates):* `clippy -D warnings`; the `slh-dsa` backend and separately bounded non-publishable HQC draft-candidate tests; the containerized, pinned-source EasyCrypt machine-check (`docker build -f formal/Dockerfile` then re-run `BindingViaCR.ec`, the explicit $K \neq \perp$ countermodel, and proof-dependency controls; the apt/opam solver and transitive graph remain non-hermetic); the C-ABI link smoke (`sh bindings/c/build-and-run.sh`); the `wasm32` and `thumbv7em` cross-builds; and the Swift/Kotlin/WASM binding tests. The WASM provider build requires `CC_wasm32_unknown_unknown` to be an absolute path to upstream LLVM Clang with the `wasm32` backend (Homebrew LLVM on macOS or `/usr/bin/clang-18` on Linux); Apple Clang is rejected. *Tier 3 (hardware-dependent):* the bare-metal time-to-session (`sudo env QPERIAPT_BARE_METAL_CONFIRMED=1 sh camera-ready-bare-metal.sh`; a frozen root-owned host-state launcher that runs all build and measured code in a clean, read-only snapshot under a dedicated locked, group/capability-cleared `no_new_privs` account and per-command cgroup v2 with finite process/memory limits on a quiesced Linux host. Each build receives a fresh Cargo home populated only from `Cargo.lock`-checksum-verified crate archives and runs in a network namespace without host networking. The harness then emits a raw hash-bound run-id bundle, including canonical before/active/after host-state metadata, only after verified host restoration; this producer-origin lane trusts the pinned dependency/toolchain closure and is not a hostile-builder or formal source-to-binary refinement proof); the source→binary CT discriminator (`sh ctstats/scripts/ct-gap-probe.sh`; x86-64/aarch64 Linux + Valgrind); the Tamarin/ProVerif models (`make` under `formal/{tamarin,proverif}`); and the platform footprint (`sh paper/footprint.sh`). The figures rebuild via `make -C paper/figures`.

REFERENCES

[1] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," 2024.

[2] C. Cremers, A. Dax, and N. Medinger, "Keeping Up with the KEMs: Binding Security of Key Encapsulation Mechanisms," *ACM CCS*, 2024. (ePrint 2023/1933.)

[3] S. Schmieg, "Unbindable Kemmy Schmidt: ML-KEM is neither MAL-BIND-K-CT nor MAL-BIND-K-PK," IACR ePrint 2024/523, 2024.

[4] J. Krämer, P. Struck, and M. Weishäupl, "Binding Security of Combined KEMs: An Analysis of Real-World KEM Combiners," IACR ePrint 2025/1416, 2025.

[5] D. Connolly, P. Schwabe, and B. Westerbaan, "X-Wing: The Hybrid KEM You've Been Looking For," draft-connolly-cfrg-xwing-kem-10, individual Internet-Draft (work in progress), 2026.

[6] D. J. Bernstein, K. Bhargavan, S. Bhasin, A. Chattopadhyay, T. K. Chia, M. J. Kannwischer, F. Kiefer, T. B. Paiva, P. Ravi, and G. Tamvada, "KyberSlash: Exploiting Secret-Dependent Division Timings in Kyber Implementations," *IACR TCHES*, 2025.

[7] M. Bellare and V. T. Hoang, "Efficient Schemes for Committing Authenticated Encryption," *EUROCRYPT*, 2022.

[8] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard," 2024.

[9] National Institute of Standards and Technology, "FIPS 205: Stateless Hash-Based Digital Signature Standard," 2024.

[10] J. W. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," *IEEE EuroS&P*, 2018.

[11] G. Alagic *et al.*, "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," NIST IR 8545, 2025.

[12] E. Fujisaki and T. Okamoto, "Secure Integration of Asymmetric and Symmetric Encryption Schemes," *CRYPTO*, 1999.

[13] D. Hofheinz, K. Hövelmanns, and E. Kiltz, "A Modular Analysis of the Fujisaki–Okamoto Transformation," *TCC*, 2017.

[14] A. Langley, M. Hamburg, and S. Turner, "Elliptic Curves for Security," RFC 7748, IETF, 2016.

[15] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 9846, IETF, July 2026.

[16] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid Key Exchange in TLS 1.3," RFC 9954, IETF, July 2026.

[17] D. Connolly, R. Barnes, and P. Grubbs, "Hybrid PQ/T Key Encapsulation Mechanisms," draft-irtf-cfrg-hybrid-kems-12, IRTF CFRG (work in progress), 2026.

[18] E. Barker *et al.*, "Considerations for Achieving Crypto Agility: Strategies and Practices," NIST CSWP 39upd1, updated June 2026.

[19] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, "Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3," RFC 10024, IETF, August 2026.

[20] K. Bhargavan, C. Jacomme, F. Kiefer, and R. Schmidt, "Formal Verification of the PQXDH Post-Quantum Key Agreement Protocol," *IEEE S&P*, 2024.

[21] Apple Security Engineering, "iMessage with PQ3: The New State of the Art in Quantum-Secure Messaging," Apple Security Research, 2024.

[22] T. Perrin, M. Marlinspike, and R. Schmidt, "The Double Ratchet Algorithm," revision 4, Signal specifications, 2025. [Online]. Available: https://signal.org/docs/specifications/doubleratchet/

[23] G. Connell and R. Schmidt, "The ML-KEM Braid Protocol," Signal specifications, 2025. [Online]. Available: https://signal.org/docs/specifications/mlkembraid/

[24] M. Marlinspike and T. Perrin, "The Sesame Algorithm: Session Management for Asynchronous Message Encryption," revision 2, Signal specifications, 2017. [Online]. Available: https://signal.org/docs/specifications/sesame/

[25] G. Connell and R. Schmidt, "Signal Protocol and Post-Quantum Ratchets," Signal, Oct. 2025. [Online]. Available: https://signal.org/blog/spqr/

[26] J. Krämer, P. Struck, and M. Weishäupl, "Binding Security of Implicitly-Rejecting KEMs and Application to BIKE and HQC," IACR ePrint 2024/1233, 2024.

[27] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, "HACL*: A Verified Modern Cryptographic Library," *ACM CCS*, 2017.

[28] Cryspen, "Verified ML-KEM (Kyber) in `libcrux`," 2024.

[29] J. B. Almeida, S. Arranz Olmos, M. Barbosa, G. Barthe, F. Dupressoir, B. Grégoire, V. Laporte, J.-C. Léchenet, C. Low, T. Oliveira, H. Pacheco, M. Quaresma, P. Schwabe, and P.-Y. Strub, "Formally Verifying Kyber: Episode V," *CRYPTO*, 2024.

[30] M. Meijers and D. Connolly, "KEM Binding in EasyCrypt," SandboxAQ software artifact: machine-checked LEAK-BIND for ML-KEM and the Fujisaki–Okamoto transform, GitHub `sandbox-quantum/EasyCrypt-KEMs`, 2024–2025. [Online]. Available: https://github.com/sandbox-quantum/EasyCrypt-KEMs

[31] G. Barthe, B. Grégoire, S. Heraud, and S. Z. Béguelin, "Computer-Aided Security Proofs for the Working Cryptographer," *CRYPTO*, 2011.

[32] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," *CAV*, 2013.

[33] B. Blanchet, "Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif," *Found. Trends Priv. Secur.*, 2016.

[34] O. Reparaz, J. Balasch, and I. Verbauwhede, "Dude, is My Code Constant Time?" *DATE*, 2017.

[35] N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," *ACM PLDI*, 2007.

[36] A. Langley, "ctgrind: Checking that Functions are Constant Time with Valgrind," 2010.

[37] National Institute of Standards and Technology, "Automated Cryptographic Validation Protocol (ACVP)," https://github.com/usnistgov/ACVP, 2024.

[38] J. Birr-Pixton *et al.*, "rustls: A Modern TLS Library in Rust," https://github.com/rustls/rustls, 2024.